

سند فنی سامانه معاونت پارلمانی ریاست جمهوری با معماری Clean و JWT

1. مقدمه

این سند فنی، طراحی و ساختار یک اپلیکیشن وب مبتنی بر Laravel با معماری Clean Architecture را تشریح می‌کند. هدف این معماری، ایجاد یک ساختار مقیاس‌پذیر، قابل نگهداری و قابل تست است. با جداسازی مسئولیت‌ها در لایه‌های منطقی، تغییرات در آینده آسان‌تر، تست واحد ساده‌تر و توسعه سریع‌تر امکان‌پذیر خواهد بود. این اپلیکیشن از JWT (JSON Web Token) برای احراز هویت و مدیریت دسترسی کاربران استفاده می‌کند تا امنیت داده‌ها و API‌ها تضمین شود.

2. اهداف

- ایجاد معماری مقیاس‌پذیر و قابل نگهداری
- جداسازی منطق دامنه از زیرساخت
- امنیت و کنترل دسترسی با JWT
- ساده‌سازی توسعه و تست واحد (Unit Test)
- انعطاف‌پذیری در انتخاب زیرساخت

3. معماری اپلیکیشن

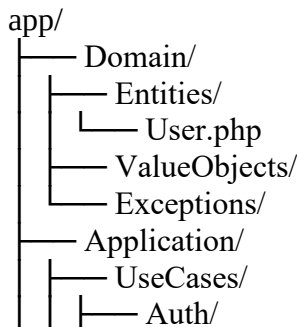
3.1 لایه‌ها

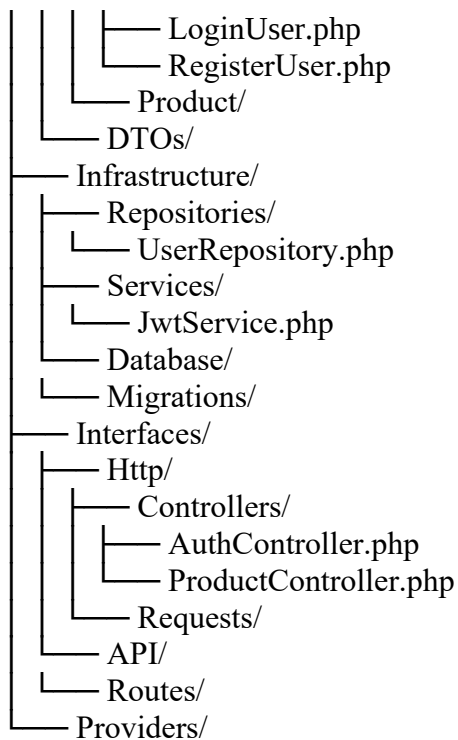
- Entity / Domain Layer: شامل Framework و وجودیت‌ها و ارزش‌ها و منطق اصلی کسب‌وکار، مستقل از DB
- Application / Use Case Layer: UI و DB سناریوهای استفاده، مستقل از
- Interface / Adapter Layer: Presentation و زیرساخت یا Application رابط بین
- Infrastructure / Framework Layer: Repository پیاده‌سازی، Service، DB، Mail، Cache و JWT

3.2 جریان داده‌ها

[Controller / Route] → [Use Case / Application] → [Entity / Domain] → [Repository / Infrastructure] → [Database / External Service]

4. ساختار پروژه در لاراول





5. JWT و احراز هویت

- نصب پکیج `tymon/jwt-auth` :
- تنظیمات فایل `config/jwt.php`
- `Middleware` برای بررسی JWT

6. نمونه API ها

Method	Endpoint	Description	Auth Required
POST	/api/register	ثبت نام کاربر	No
POST	/api/login	ورود کاربر	No
POST	/api/logout	خروج کاربر	Yes

مدیریت محصول

Method	Endpoint	Description	Auth Required
GET	/api/products	دریافت لیست محصولات	Yes
POST	/api/products	ایجاد محصول جدید	Yes
PUT	/api/products/{id}	بروزرسانی محصول	Yes

Method	Endpoint	Description	Auth Required
DELETE	/api/products/{id}	حذف محصول	Yes

7. پایگاه داده

- استفاده از MySQL/PostgreSQL
- جداول اصلی: users, products, orders, roles, permissions

8. نکات امنیتی

- رمزنگاری پسورد با bcrypt
- محدودیت تعداد تلاش ورود (Rate Limiting)
- JWT با تاریخ انقضا و Refresh Token
- استفاده از HTTPS
- Centralized Error Handling

9. تست و توسعه

- Unit Test
- Factory و Seeder برای داده‌های تستی

10. راهکارهای بهینه پیاده سازی شده

- جداسازی کامل لایه‌ها و Dependency Injection
- استفاده از DTO ها برای انتقال داده
- استفاده از Repository Interface برای جداسازی دیتابیس
- Centralized Exception Handling
- رعایت اصول SOLID و Clean Code